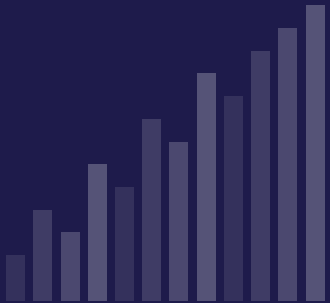
AULA 22 · BLOCO 3: DICIONÁRIOS: HASH E ÁRVORES

# Árvores binárias de busca: percursos e consultas

Cormen et al., *Algoritmos*: seções 12.1 e 12.2



TEORIA

# Roteiro da aula

## Ao final, você será capaz de

- Enunciar a propriedade de árvore binária de busca
- Percorrer em ordem, pré-ordem e pós-ordem
- Buscar, achar mínimo, máximo, sucessor e predecessor em $O(h)$
- Medir o custo médio de busca em árvores aleatórias: $\approx 1{,}39 \lg n$

### Teoria

Propriedade, percursos, consultas e o papel da altura

### Prática em C

ABB em C: inserção, percurso, busca, mínimo, máximo e sucessor

### Experimento

Custo médio de busca contra $2(1 + 1/n)H_n - 3$

### Conexões

Sistemas de arquivos, compiladores, bancos de dados e geometria



# Teoria

- Cormen et al., seções 12.1 e 12.2
- A propriedade de árvore binária de busca
- Percurso em ordem
- Busca, mínimo, máximo e sucessor

TEORIA

# A propriedade de árvore binária de busca

- Cada nó $x$ tem *chave*, *esq*, *dir* e *pai*.
- **Propriedade:** se $y$ está na subárvore esquerda de $x$, $y.chave \leq x.chave$; se está na direita, $y.chave \geq x.chave$.
- As operações custam $O(h)$, onde $h$ é a **altura**.
- $h$ vai de $\lfloor \lg n \rfloor$ (árvore cheia) a $n - 1$ (uma corrente).

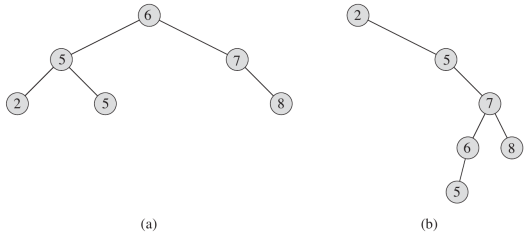



**Figura 12.1** de Cormen et al. (2009) — duas ABBs com as mesmas chaves: (a) altura 2; (b) altura 4, menos eficiente.

TEORIA

# Percurso em ordem

- Imprime as chaves **em ordem**: consequência direta da propriedade.
- **Teorema 12.1:** $\Theta(n)$ para $n$ nós.
- Pré-ordem: raiz antes das subárvores (copiar a árvore).
- Pós-ordem: raiz depois (liberar a memória, avaliar expressões).

**INORDER-TREE-WALK($x$)**

```
1 se x ≠ NIL então
2     INORDER-TREE-WALK(x.esq)
3     imprima x.chave
4     INORDER-TREE-WALK(x.dir)
```

TEORIA

# Busca, mínimo, máximo e sucessor

- TREE-SEARCH: desce à esquerda ou à direita conforme a chave: $O(h)$.
- TREE-MINIMUM: sempre à esquerda. TREE-MAXIMUM: sempre à direita.
- TREE-SUCCESSOR($x$): se há subárvore direita, o mínimo dela; senão, suba até chegar por um filho esquerdo.
- **Teorema 12.2:** todas $O(h)$.

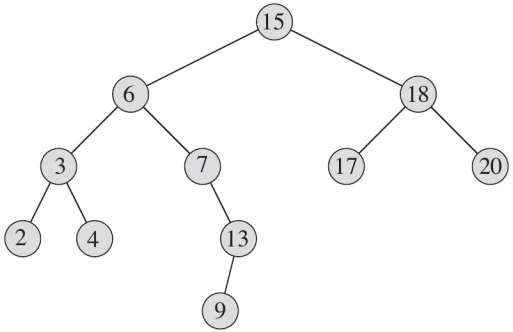



**Figura 12.2** de Cormen et al. (2009) — busca por 13: $15 \to 6 \to 7 \to 13$. Mínimo: sempre à esquerda (2).

# Prática em C

- Nós com ponteiro para o pai (necessário para o sucessor)
- Percurso em ordem recursivo; busca iterativa
- Teste com as chaves da Figura 12.2

PRÁTICA EM C

# Árvore binária de busca em C (1/4)

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct No { int chave; struct No *esq, *dir, *pai; } No;

No *insere(No *raiz, int k) {                  // TREE-INSERT (detalhado na aula 23)
    No *z = calloc(1, sizeof(No)), *y = NULL, *x = raiz;
    z->chave = k;
    while (x) { y = x; x = k < x->chave ? x->esq : x->dir; }
    z->pai = y;
    if (!y) return z;
    if (k < y->chave) y->esq = z; else y->dir = z;
    return raiz;
}
```

PRÁTICA EM C

# Árvore binária de busca em C (2/4)

```c

// INORDER-TREE-WALK: Θ(n), chaves em ordem
void em_ordem(No *x) {
    if (!x) return;
    em_ordem(x->esq);
    printf(" %d", x->chave);
    em_ordem(x->dir);
}

No *busca(No *x, int k) {                    // ITERATIVE-TREE-SEARCH: O(h)
    while (x && k != x->chave) x = k < x->chave ? x->esq : x->dir;
    return x;
}
```

PRÁTICA EM C

# Árvore binária de busca em C (3/4)

```
0
No *minimo(No *x) { while (x->esq) x = x->esq; return x; }
No *maximo(No *x) { while (x->dir) x = x->dir; return x; }

No *sucessor(No *x) {                          // TREE-SUCCESSOR: O(h)
    if (x->dir) return minimo(x->dir);         // caso 1: menor da subárvore direita
    No *y = x->pai;                            // caso 2: sobe até virar filho esquerdo
    while (y && x == y->dir) { x = y; y = y->pai; }
    return y;
}

void libera(No *x) { if (x) { libera(x->esq); libera(x->dir); free(x); } }
```

PRÁTICA EM C

# Árvore binária de busca em C (4/4)

```
0
int main(void) {
    No *raiz = NULL;
    int k, ultimo = 0;
    while (scanf("%d", &k) == 1) { raiz = insere(raiz, k); ultimo = k; }
    if (!raiz) return 1;
    printf("em ordem:");
    em_ordem(raiz);
    printf("\nmínimo %d, máximo %d\n", minimo(raiz)->chave, maximo(raiz)->chave);
    No *x = busca(raiz, 13), *s = x ? sucessor(x) : NULL;
    if (s) printf("sucessor de 13: %d\n", s->chave);
    printf("última chave lida (%d) %s\n", ultimo,
           busca(raiz, ultimo) ? "encontrada" : "ausente");
    libera(raiz);
    return 0;
}
```



# Experimento

- Hipótese: $2(1+1/n)H_n - 3 \approx 1{,}39 \lg n$
- $n = 2^{10}$ a $2^{21}$, média de várias árvores
- Custo de buscar uma chave = sua profundidade + 1

EXPERIMENTO

# Quanto custa buscar numa árvore aleatória?

**Hipótese.** Inserir chaves em ordem aleatória equivale a montar a árvore de recursão do quicksort: cada chave é comparada com seus ancestrais. O custo médio de uma busca com sucesso é

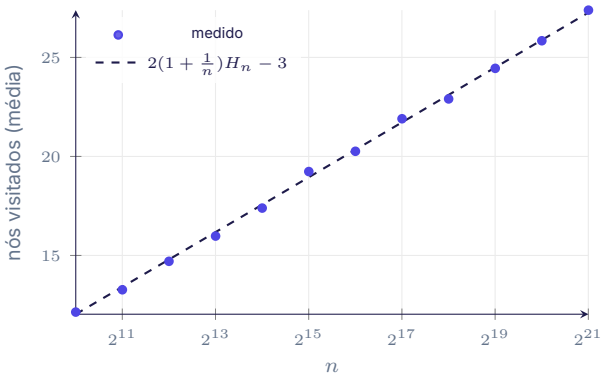
$$2\left(1+\frac{1}{n}\right)H_n - 3 \approx 2\ln n \approx 1{,}39\lg n.$$

**Método**

1. Chaves de 32 bits sorteadas (xorshift)
2. Profundidade de cada nó registrada na inserção
3. $n = 2^{10}, \ldots, 2^{21}$
4. Média de 10 árvores (3 para $n > 2^{17}$)

EXPERIMENTO

# Nós visitados por busca




| $n$ | medido | razão |
|---|---|---|
| 1.024 | 12,1 | 1,009 |
| 4.096 | 14,7 | 0,994 |
| 16.384 | 17,4 | 0,990 |
| 65.536 | 20,3 | 0,996 |
| 262.144 | 22,9 | 0,991 |
| 2.097.152 | 27,4 | 1,004 |

razão = medido $\div\, 2(1+\frac{1}{n})H_n - 3$

✓ desvio máx. 0,9%

Com $n$ em escala logarítmica, o custo é uma reta: cresce como $\lg n$, cerca de 1,39 nó a cada vez que $n$ dobra.

EXPERIMENTO

# O que o experimento mostrou

O custo médio segue $2(1 + 1/n)H_n - 3$ com desvio abaixo de 1%: para dois milhões de chaves, 27 nós por busca. Uma árvore aleatória é só 39% mais funda (em média) que uma perfeitamente balanceada. O perigo está nas entradas **não** aleatórias (próxima aula).

Reproduza: `gcc -O2 aula22_experimento.c -o exp -lm && ./exp`



# Conexões

- Sistemas de arquivos
- Compiladores
- Bancos de dados
- Geometria computacional

CONEXÕES

# Onde isso aparece na computação

### Sistemas de arquivos

Diretórios são árvores; ext4 e Btrfs indexam nomes em árvores de busca para achar um arquivo entre milhões sem varrer todos.

### Compiladores

Árvores sintáticas são percorridas em pós-ordem para gerar código e avaliar expressões; o percurso em ordem imprime a expressão infixa.

### Bancos de dados

Consultas por intervalo (`BETWEEN`) usam o sucessor: acha-se o início e anda-se em ordem. Hash não faz isso; árvores fazem.

### Geometria computacional

Árvores k-d e de intervalos generalizam a ABB para pontos no plano: "quais restaurantes estão neste retângulo do mapa?.

PARA CASA

# Para casa

## Exercícios

1. (12.1-1) Desenhe ABBs de alturas 2, 3, 4, 5 e 6 com as chaves $\{1, 4, 5, 10, 16, 17, 21\}$.
2. (12.1-3) Escreva o percurso em ordem não recursivo, com uma pilha.
3. (12.2-1) Quais sequências de busca por 363 são impossíveis numa ABB? Por quê?
4. (12.2-3) Escreva Tree-Predecessor.
5. Altere o experimento para medir a busca **sem** sucesso. A fórmula muda?

**Leitura**
Cormen et al., seções 12.1 e 12.2.

**Código**
josedihego.net/estruturas